

Drone Delivery Monitoring & Failure Analytics Pipeline

Project Report

1. Problem Statement

Drone-based delivery is being adopted for last-mile logistics because of its speed and automation, but fleets face recurring operational problems:

- Battery limitations and energy management
- Unpredictable weather affecting flight safety
- GPS signal instability in dense urban environments
- No centralized system to monitor fleet health

Most organizations run drones without a data-driven pipeline to track performance, catch anomalies, or optimize routes. This project builds that pipeline: it ingests raw drone logs, cleans and enriches the data, and produces the KPIs stakeholders need to act on.

2. Objectives

- Track drone deliveries and operational health
- Analyze failure patterns and root causes
- Identify high-risk geographical delivery zones
- Improve delivery efficiency and resource utilization

3. Architecture (Medallion Pattern)

The pipeline follows the Medallion Architecture, moving data through four progressively cleaner layers:

```
Raw CSV files
|
v
BRONZE - raw ingestion, original values preserved, ingestion
metadata (ingestion_time, source_file) added
|
v
SILVER - cleaning, type casting, null/duplicate handling, joins
across tables, derived columns (delivery_duration,
failure_flag)
|
v
GOLD - aggregated, business-ready KPI tables
|
v
SQL INSIGHTS - BI-facing queries answering business questions
```

Why this pattern: each layer has one job, so a bug in cleaning logic never corrupts the raw record, and a bad KPI calculation never forces a full re-ingest. Bronze is the permanent, replayable source of truth; everything downstream can be rebuilt from it at any time.

4. Technology Stack

- **PySpark 3.5.1** – distributed data processing
- **Parquet** – columnar storage per layer (Delta Lake is the production target; see Section 9)
- **Spark SQL** – the BI / reporting layer
- **Python (csv module)** – synthetic sample data generation

5. Datasets

5.1 drones.csv

drone_id (INT), model (STRING), max_range_km (FLOAT) — 25 sample drones across 5 models

5.2 deliveries.csv

delivery_id (INT), drone_id (INT), source (STRING), destination (STRING), distance_km (FLOAT), start_time (TIMESTAMP), end_time (TIMESTAMP), battery_consumed (FLOAT) — 600 sample deliveries across 10 delivery zones

5.3 flight_logs.csv

log_id (INT), delivery_id (INT), drone_id (INT), battery_level (FLOAT), gps_signal (FLOAT), status (STRING) — 600 sample outcomes: SUCCESS, FAILED_BATTERY, FAILED_WEATHER, FAILED_SIGNAL

6. KPI Derivation Logic

#	KPI	Formula	Grouped by
1	Delivery Success Rate	(SUCCESS count / Total Flights) x 100	drone
2	Failure Rate	(FAILED_* count / Total Flights) x 100	drone
3	Avg Delivery Time	AVG(end_time - start_time)	drone
4	Battery Efficiency	distance_km / battery_consumed	drone (avg)
5	High-Risk Zones	COUNT(failure_flag = 1)	destination

7. Pipeline Components Built

File	Purpose
generate_sample_data.py	Creates the three raw CSVs
bronze_layer.py	Raw ingestion, adds ingestion metadata
silver_layer.py	Cleans, types, dedupes, joins, derives columns

gold_layer.py	Computes all 5 KPI tables
sql_insights.sql	The 6 business questions as plain SQL

Data quality rules applied in Silver:

- Rows missing a primary key or required field are dropped
- Duplicate primary keys are removed, keeping one record
- Timestamps are cast to proper `TIMESTAMP` type
- Numeric fields are cast to `FLOAT` for consistent aggregation

8. Results From a Test Run (Sample Data)

Layer	Table	Row Count
Bronze	drones / deliveries / flight_logs	25 / 600 / 600
Silver	drones / deliveries / flight_logs	25 / 600 / 600
Gold	drone_kpis	25
Gold	avg_delivery_time	25
Gold	battery_efficiency	25
Gold	high_risk_zones	10

Sample business findings from this test run:

- The two highest-risk delivery zones (most failures) were Hillcrest and Northgate.
- The worst-performing drone by failure rate was drone_id 9 (model CargoWing), which also had an above-average delivery time — flagged by the “needs attention first” SQL query as a combined reliability and speed problem.
- Fleet-wide average success rate was computed directly from the Gold drone_kpis table via a single SQL aggregate query.

These numbers come from randomly generated sample data used to prove the pipeline end-to-end. Swap in real fleet logs by replacing the three raw CSVs with the same schema.

9. Note on Delta Lake

The original design specifies Delta Lake for ACID transactions, time travel, and schema enforcement. The pipeline was built and tested using Parquet as a stand-in, because Delta Lake's core JARs are fetched from Maven Central at runtime, and the build/test environment used for this project did not have Maven access.

The pipeline logic is unchanged either way. To use real Delta Lake (e.g. on Databricks, which has Delta natively):

- Change `.format("parquet")` to `.format("delta")` in every write
- Change `spark.read.parquet(...)` to `spark.read.format("delta").load(...)` in every read

10. How to Run

Locally

```
pip install pyspark==3.5.1
python3 src/generate_sample_data.py
python3 src/bronze_layer.py
python3 src/silver_layer.py
python3 src/gold_layer.py
```

Run the four scripts in order: sample data generation → Bronze → Silver → Gold. Each script reads the previous layer's output and writes its own layer under data/bronze, data/silver, and data/gold.

On Databricks

- Upload the project folder to a Repo or Workspace folder.
- Update the hardcoded data paths at the top of each script to a Unity Catalog Volume (/Volumes/<catalog>/<schema>/<volume>/...) or DBFS (/dbfs/FileStore/...) rather than a Workspace path, since Workspace storage doesn't support distributed Spark writes.
- Run bronze_layer.py, then silver_layer.py, then gold_layer.py — either as notebook cells or as sequential job tasks.

11. Conclusion

This project delivers a working, end-to-end data engineering pipeline for the drone logistics domain. Using PySpark and a layered (Medallion) architecture, it converts raw, ungoverned flight telemetry into governed, business-ready KPI tables and a SQL reporting layer. The result lets an organization move from reactive troubleshooting (finding out about a failure after the fact) to proactive operational optimization (spotting unreliable drones and high-risk zones before they cause repeated failures).